

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Teste de Software

Test Patterns

Aluno: Vitor Rebula Nogueira

Matricula: 820520

Disciplina: Teste de Software

Belo Horizonte - 2026

1. PADROES DE CRIACAO DE DADOS (BUILDERS)

1.1 Por que usar CarrinhoBuilder em vez de CarrinhoMother?

O padrao **Object Mother** centraliza a criacao de objetos por meio de metodos estaticos que retornam instancias pre-configuradas e fixas. Ele e ideal quando a entidade e simples e suas variacoes entre testes sao poucas e bem definidas. No projeto, o padrao foi aplicado a entidade *User*, pois existem apenas dois perfis relevantes para os cenarios de teste: o usuario padrao e o usuario premium.

O *Carrinho*, por outro lado, e um objeto intrinsecamente composto e variavel: ele referencia um usuario, contem uma lista de itens com precos distintos e pode estar vazio ou cheio. Aplicar o padrao Object Mother a esse objeto levaria inevitavelmente a uma **explosao de metodos estaticos**, como por exemplo: `CarrinhoMother.carrinhoVazioComUsuarioPremium()` ou `CarrinhoMother.carrinhoComDoisItensEUsuarioPadrao()`, tornando a classe dificil de manter e pouco expressiva.

O padrao **Data Builder** resolve esse problema oferecendo uma API fluente que permite ao teste declarar apenas os aspectos relevantes para aquele cenario especifico, mantendo valores padrao sensatos para o restante. Dessa forma, cada teste comunica sua intencao de forma clara, sem ruido de configuracao desnecessario.

1.2 Exemplo Antes x Depois

Antes: Setup Manual (sem Builder)

Sem o padrao Builder, cada teste precisaria instanciar manualmente todos os objetos necessarios, mesmo aqueles irrelevantes para o cenario em questao:

Setup manual repetitivo (sem Builder):

```
// Teste de falha no pagamento
const user = new User(1, 'Joao Silva', 'joao@email.com', 'PADRAO');
const item = new Item('Produto Padrao', 100);
const carrinho = new Carrinho(user, [item]);

// Teste de desconto Premium
const userPremium = new User(2, 'Maria Premium', 'premium@email.com', 'PREMIUM');
const itemCaro = new Item('Produto X', 200);
const carrinhoPremium = new Carrinho(userPremium, [itemCaro]);

// Teste de carrinho vazio
const userQualquer = new User(3, 'Teste', 'teste@email.com', 'PADRAO');
const carrinhoVazio = new Carrinho(userQualquer, []);
```

Esse padrao de setup repete informacoes irrelevantes para cada cenario (id do usuario, nome, construtores de itens) e acopla os testes a detalhes internos das classes de dominio. Qualquer mudanca no construtor de *User* ou *Carrinho* exigiria atualizacao em multiplos arquivos de teste, caracterizando o *Test Smell* conhecido como **Obscure Setup**.

Depois: Setup com Data Builder

Com o *CarrinhoBuilder* implementado, cada teste declara apenas o que e relevante:

Setup com Builder: limpo e expressivo:

```
// Teste de falha - apenas o carrinho padrao e necessario
const carrinho = new CarrinhoBuilder().build();

// Teste de desconto Premium - intencao explicita e legivel
const usuarioPremium = UserMother.umUsuarioPremium();
const carrinho = new CarrinhoBuilder()
  .comUser(usuarioPremium)
  .comItens([new Item('Produto X', 200)])
  .build();

// Teste de carrinho vazio - expressivo e direto
const carrinho = new CarrinhoBuilder().vazio().build();
```

1.3 Como o Builder melhora a legibilidade e a manutencao

Criterio	Setup Manual	Data Builder
Legibilidade	Mistura dados irrelevantes com os do cenario	Declara apenas o que importa para o teste

Criterio	Setup Manual	Data Builder
Manutencao	Mudancas no dominio afetam todos os testes	Mudancas absorvidas pelo Builder centralizado
Reutilizacao	Codigo duplicado em varios arquivos	Logica de criacao centralizada em um unico lugar
Expressividade	A intencao fica escondida no setup	A intencao e explicita na cadeia de metodos fluentes

Em termos praticos, ao adicionar um novo campo obrigatorio ao construtor de *Carrinho*, bastaria atualizar o `CarrinhoBuilder` e todos os testes que o utilizam continuariam funcionando sem nenhuma modificacao individual.

2. PADROES DE TEST DOUBLES (MOCKS VS. STUBS)

2.1 Identificacao das dependencias no teste Premium

O teste da Etapa 5, "*quando um cliente Premium finaliza a compra*", envolve tres dependencias externas injetadas no CheckoutService. Cada uma recebe um tipo diferente de duple, conforme sua funcao no cenario:

Dependencia	Tipo de Duple	Finalidade no teste
GatewayPagamento	Stub	Retorna { success: true } para o fluxo avancar
PedidoRepository	Stub	Retorna pedido ficticio para simular persistencia
EmailService	Mock	Verifica se o e-mail foi enviado com os argumentos corretos

Codigo do teste (Arrange e Assert):

```
// Arrange
const gatewayStub = { cobrar: jest.fn().mockResolvedValue({ success: true }) };
const repositoryStub = { salvar: jest.fn().mockResolvedValue(pedidoSalvo) };
const emailMock = { enviarEmail: jest.fn().mockResolvedValue(undefined) };

// Assert - Verificacao de Estado (Stub)
expect(gatewayStub.cobrar).toHaveBeenCalledWith(180, {});

// Assert - Verificacao de Comportamento (Mock)
expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);
expect(emailMock.enviarEmail).toHaveBeenCalledWith(
  'premium@email.com',
  'Seu Pedido foi Aprovado!',
  expect.any(String)
);
```

2.2 Por que GatewayPagamento e um Stub?

O GatewayPagamento funciona como um **Stub** porque seu papel no cenario de teste e exclusivamente **controlar o estado do fluxo**: ao retornar { success: true }, ele permite que o CheckoutService avance para as etapas de persistencia e notificacao. A preocupacao central do teste nao e se o gateway foi chamado, mas sim *qual valor* foi

passado a ele (o total com desconto de 10%) e *qual estado* o serviço produz a partir dessa resposta.

Esse enfoque é chamado de **Verificação de Estado** (*State Verification*): o teste valida o resultado observável da execução, ou seja, que o método cobrar foi invocado com o valor correto de R\$ 180,00, comprovando que a regra de desconto para clientes Premium foi aplicada corretamente.

Definição: Um Stub fornece respostas pre-programadas as chamadas feitas durante o teste e não realiza verificações sobre como foi chamado. Seu objetivo é colocar o sistema sob teste em um estado específico para que o comportamento possa ser avaliado.

2.3 Por que EmailService é um Mock?

O EmailService é tratado como um **Mock** porque o objetivo do teste é verificar um **efeito colateral observável**: o serviço de checkout deve enviar um e-mail de confirmação ao usuário e esse envio não produz nenhum valor de retorno que possa ser inspecionado pelo método processarPedido.

Como o e-mail é um efeito colateral sem estado observável no resultado final do método, a única forma de verificar que ele ocorreu é inspecionar a **interação** com a dependência: quantas vezes enviarEmail foi chamado e com quais argumentos. Esse enfoque é chamado de **Verificação de Comportamento** (*Behavior Verification*).

Definição: Um Mock é um objeto pre-programado com expectativas sobre as chamadas que deve receber. Ele é usado para verificar interações: quem chamou, quantas vezes e com quais argumentos, ao invés de apenas fornecer respostas ao sistema.

A distinção prática pode ser resumida em uma pergunta: "Estou testando o que o sistema retorna, ou estou testando o que o sistema fez?" Quando a resposta é "o que retorna", usa-se um Stub. Quando a resposta é "o que fez", usa-se um Mock.

3. CONCLUSAO

A implementacao deliberada de Padroes de Teste demonstrou, na pratica, como a organizacao do codigo de teste impacta diretamente sua sustentabilidade e eficacia. O uso do padrao **Object Mother** para a entidade *User* centralizou a criacao de instancias simples e imutaveis, eliminando duplicacao. O padrao **Data Builder** para o *Carrinho*, por sua vez, resolveu o *Test Smell* de *Obscure Setup*: em vez de um conjunto confuso de instanciacoess manuais, cada teste declara apenas o que e relevante para o cenario que valida.

No campo do isolamento de dependencias, a distincao entre **Stubs** e **Mocks** revelou-se fundamental. Utilizar um Stub onde um Mock seria necessario, ou vice-versa, leva a testes que nao verificam comportamentos criticos (como o envio de e-mail) ou acoplam desnecessariamente o teste a implementacao interna do servico. A clareza dessa distincao, fundamentada nos conceitos de *State Verification* e *Behavior Verification*, e o que permite escrever testes precisos e expressivos.

Em conjunto, esses padroes contribuem para uma suite de testes que resiste a evolucao do sistema: quando o dominio muda, ajustes pontuais nos Builders ou nos doubles sao suficientes para manter a suite funcional. Testes escritos com essa disciplina nao sao um custo adicional ao desenvolvimento, mas sim uma rede de seguranca que viabiliza refatoracoes com confianca e documenta o comportamento esperado do sistema de forma viva e executavel.